

Java Interview Questions for Freshers

Q 1. What is Java?

- Java is a high-level, object-oriented programming language developed by Sun Microsystems (now owned by Oracle).

Q 2. What are Java's data types?

- Primitive: byte, short, int, long, float, double, char, boolean.
- Non-primitive: Strings, Arrays, Classes, Interfaces.

Q 3. What makes Java a platform-independent language?

Java is platform-independent because of the JVM, which converts Java bytecode into machine code specific to each operating system, allowing Java programs to run on any system with a JVM.

Q 4. Is Java considered a fully object-oriented programming language?

- Java is not considered fully object-oriented because it uses primitive data types like int, char, and boolean, which are not objects.
- Java allows the use of static members and methods, which are accessible without creating an object.
- Java supports core OOP principles like encapsulation, inheritance, polymorphism, and abstraction, making it primarily object-oriented but not purely so.

Q 5. What are the key features of Java?

- Platform-independent, Object-oriented, Secure, Robust, Multithreaded, Distributed, Portable, High-performance.

Q 6. What is the JVM?

- JVM (Java Virtual Machine) is a virtual machine that runs Java bytecode and makes Java platform-independent.

Q 7. What is JDK, JRE, and JVM?

- JDK: Java Development Kit - includes JRE and development tools.
- JRE: Java Runtime Environment - provides libraries and JVM.
- JVM: Executes Java bytecode.

Q 8. Instance Variable vs Local Variable

Instance Variables

- Instance variables are declared within a class but outside of any methods, constructors, or blocks.
- They are associated with a particular instance of the class and have default values if not initialized.
- Instance variables are accessible by all methods in the class and retain their values as long as the object exists.

Local Variables

- Local variables, on the other hand, are declared inside a method, constructor, or block.
- Local variables do not have default values and must be explicitly initialized before use.
- They are accessible only within the method, constructor, or block where they are declared.

Q 9. What is a Class and an Object?

- **Class:**

- A blueprint to create objects.
- It defines the properties (fields/variables) and behaviors (methods/functions) that the objects created from the class will have.
- A class is not a real object itself but defines how objects will be structured.

- **Object:**

- An instance of a class.
- Objects are created based on the structure defined in the class and represent real-world entities or data
- Objects store actual data in their fields and can perform operations using the methods defined in the class.

Q 10. What is the difference between == and equals()?

- == checks reference equality.
- equals() checks value equality.

Q 11. What are constructors in Java?

- Special methods used to initialize objects.
- They have the same name as the class and no return type (not even void).
- Constructors are called when an object of the class is created, ensuring the object is set up with initial values.

Q 12. What are access modifiers in Java?

- Access modifiers determine the level of access control to the members (fields, methods, and inner classes) of a class.
- **Types of Access Modifiers:**
 - public : Members are accessible from any class, regardless of package.
 - private: Members are accessible only within the same class.
 - protected: Members are accessible within the same package or by subclasses (even if they are in different packages).
 - default: If no modifier is specified, the member is accessible only within the same package.

Q 13. What is Polymorphism in Java?

- Polymorphism allows methods to perform different tasks based on the object calling them.
- Two types: Compile-time (method overloading) and Runtime (method overriding) polymorphism.

Q 14. Explain the concept of Inheritance.

- Inheritance allows a class to acquire properties and methods of another class.
- Promotes code reusability and establishes a parent-child relationship between classes.
- Enables the creation of hierarchical classifications.

Q 15. What is Abstraction?

- Abstraction is the concept of hiding the implementation details and showing only the necessary details to the user.
- Achieved using abstract classes and interfaces.

Q 16. What is Encapsulation?

- Encapsulation is bundling data and methods that operate on the data within one unit, typically a class.
- Provides control over the data by restricting access through access modifiers (private, public, protected).
- Enhances security by hiding the internal state of the object.

Q 17. Difference between Abstract Class and Interface.

Abstract Class:

- Can have both abstract and non-abstract methods.
- Can include fields and constructors.
- Supports single inheritance.

Interface:

- Contains only abstract methods (until Java 8, later versions can have default and static methods).
- No fields allowed (except constants).
- Supports multiple inheritance.

Q 18. What is the difference between String, StringBuilder, and StringBuffer?

String

- **Immutable:** Once created, the content of a String cannot be changed. Any modification creates a new String.
- **Storage:** Stored in the **String pool** (if created using string literals), which allows reusability and memory efficiency.

- **Performance:** Due to immutability, operations like concatenation involve creating new Strings, which can lead to performance overhead in situations involving frequent modifications.
- **Usage:** Best suited for situations where the value does not change frequently or when working with small amounts of data.

StringBuilder

- **Mutable:** Can be modified after creation, allowing efficient operations such as append, insert, delete, and replace.
- **Performance:** Faster than StringBuffer in single-threaded operations due to the absence of synchronization overhead.
- **Usage:** Ideal for situations where frequent modifications to strings are required, particularly in a single-threaded environment.

StringBuffer

- **Mutable:** Can be modified after creation, similar to StringBuilder.
- **Performance:** Slower than StringBuilder because of the synchronization overhead, making it less efficient in single-threaded operations.

Q 19. Explain this and super keywords.

this:

- Refers to the current instance of the class.
- Used to differentiate between class variables and parameters with the same name.

super:

- Refers to the immediate parent class instance.
- Used to access parent class methods, constructors, and fields.

Q 20. What are Java Collections?

- A framework that provides an architecture to store and manipulate groups of objects.
- Includes interfaces like List, Set, and Map, and their implementations like ArrayList, HashSet, and HashMap.
- Supports operations like searching, sorting, insertion, manipulation, and deletion.

Q 21. Explain Exception Handling.

- A mechanism to handle runtime errors in a controlled manner.
- Involves the use of try, catch, finally, throw, and throws.
- Ensures the program runs smoothly even when an error occurs, by catching and managing exceptions.

Q 22. Difference between throw and throws.

- **throw:**
 - Used to explicitly throw an exception.
 - Syntax: `throw new ExceptionType("Error message");`
 - Follows the exception object.
 - Typically used inside a method or a block of code.
 - Only one exception can be thrown at a time.
- **throws:**
 - Declares that a method can throw one or more exceptions.

- Syntax: methodName() throws ExceptionType1, ExceptionType2 { ... }
- Placed in the method signature.
- Alerts the caller of the method about the potential exceptions it might throw.
- Multiple exceptions can be declared separated by commas.

Q 23. Can we use catch without trying?

- No, you cannot use catch without trying.
 - catch is designed to handle exceptions that occur in the try block.
 - The syntax requires that a catch block must follow a try block.

Q 24. Can we use multiple catch blocks with one try block?

- Yes, you can use multiple catch blocks with a single try block.
 - This allows handling different types of exceptions separately.
 - Each catch block can handle a specific exception type.

Syntax:

```
try {

    // Code that may throw exceptions

} catch (ExceptionType1 e) {

    // Handle ExceptionType1

} catch (ExceptionType2 e) {
```

```
        // Handle ExceptionType2  
    }
```

Q 25. What is JIT Compile?

- **JIT Compiler** is part of the JVM that converts bytecode into native machine code at runtime.
- **Purpose:** Improves performance by avoiding repeated interpretation of bytecode.
- **How it Works:**
 - Detects frequently used code (hotspots).
 - Compiles these hotspots into native code for faster execution.
- **Advantages:**
 - Speeds up execution.
 - Performs runtime optimizations.
 - Adaptive to actual usage patterns.

Q 26. Explain the use of the final keyword in variable, method and class.

- **final Keyword with Variables:**
 - **Use:** To declare constants.
 - **Behavior:**
 - Once a variable is assigned a value, it cannot be changed.
 - Must be initialized at the time of declaration or within a constructor.
- **Example:**
final int MAX_VALUE = 100;

- **final Keyword with Methods:**

- **Use:** To prevent method overriding.
- **Behavior:**
 - A final method cannot be overridden by subclasses.
 - Ensures the method's behavior remains consistent across different subclasses.

- **Example:**

```
class Parent {  
  
    final void display() {  
  
        System.out.println("This is a final method.");  
  
    }  
  
}
```

- **final Keyword with Classes:**

- **Use:** To prevent inheritance.
- **Behavior:**
 - A final class cannot be extended.
 - Ensures the class cannot be subclassed, maintaining its structure and behavior.

- **Example:**

```
final class Utility {  
  
    // Utility methods
```

}

Q 27. Difference between static methods, static variables.

- **Static Methods:**

- Static methods belong to the class, not to any specific instance of the class.
- Can be called directly using the class name without creating an object.
- Cannot access instance variables or instance methods directly.

- **Static Variables:**

- Static variables are shared among all instances of the class.
- They are initialized only once at the start, when the class is loaded.
- Can be accessed directly using the class name or through any instance.

Q 28. Interfaces vs Abstract Classes

Feature	Interface	Abstract Class
Methods	All methods are abstract (until Java 8).	Can have both abstract and concrete methods.
Fields	Only public, static, and final fields (constants).	Can have instance variables, static variables, and constants.

Multiple Inheritance	Supports multiple inheritance.	Does not support multiple inheritance (single inheritance only).
Constructors	Cannot have constructors.	Can have constructors.
Access Modifiers	Methods are implicitly public.	Methods can have any access modifier.
Implementation	Cannot have method implementation (except default and static methods from Java 8).	Can provide complete, default behavior methods.
Use	Used to define a contract for classes to implement.	Used to share common behavior among related classes.
Inheritance Keyword	Implemented using the implements keyword.	Inherited using the extends keyword.

Q 29. Java works as “pass by value” or “pass by reference” ?

Java works as "**pass by value**" for both **primitive** and **reference** types, but the behavior differs slightly between the two:

- **Pass by Value (Primitive Data Types):**

- When you pass a primitive type (e.g., `int`, `float`) to a method, a copy of the value is passed.
- Any changes to the parameter inside the method do not affect the original value.
- **Example:**

```
void modify(int num) {  
    num = 10;  
}  
  
public static void main(String[] args) {  
    int x = 5;  
    modify(x);  
    System.out.println(x); // Output: 5 (Original value is  
    unchanged)  
}
```

- **Pass by Value (Reference Types):**

- When you pass an object reference (e.g., an array or a custom object) to a method, a copy of the reference (address) is passed, not the actual object.
- You can modify the object's fields using this reference, but you cannot change the reference itself (i.e., you

cannot reassign the reference to point to a new object).

- **Example:**

```
void modifyArray(int[] arr) {  
  
    arr[0] = 10; // Changes the original array  
  
}  
  
public static void main(String[] args) {  
  
    int[] numbers = {1, 2, 3};  
  
    modifyArray(numbers);  
  
    System.out.println(numbers[0]); // Output: 10  
    (Array is modified)  
  
}
```

Q 29. Method Overloading vs Method Overriding

- **Method Overloading:**

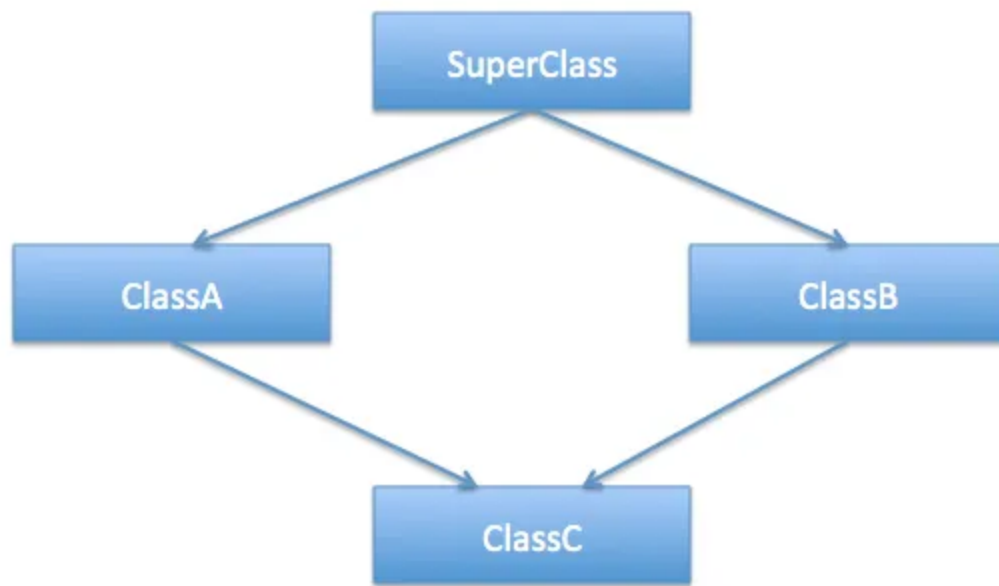
- Method overloading occurs when multiple methods in the same class have the **same name** but **different parameters** (different number or types of parameters).
- It happens at **compile-time** (also called compile-time polymorphism).
- It is based on the **number, order** or **type** of method parameters.
- It can have **different return types**, but the return type alone cannot be used to differentiate overloaded methods.

- **Method Overriding:**

- Method overriding occurs when a subclass provides its own **implementation** of a method that is already defined in its **parent class**.
- It happens at **runtime** (also called runtime polymorphism).
- The method in the subclass must have the **same name**, **same return type**, and **same parameter list** as the method in the parent class.
- It allows a subclass to provide specific behavior while retaining the parent class method's signature.

Q 30. What is the Diamond Problem in Java?

- The Diamond Problem in Java is a problem that arises in multiple inheritance scenarios. It occurs when a class inherits from two or more classes that have a method with the same signature, potentially leading to ambiguity about which method should be called.



- **Example:**

```
interface A {  
    default void display() {  
        System.out.println("Display method from interface A");  
    }  
}  
  
interface B extends A {  
    default void display() {  
        System.out.println("Display method from interface B");  
    }  
}
```

```
interface C extends A {  
    default void display() {  
        System.out.println("Display method from interface C");  
    }  
}  
  
class D implements B, C {  
    // Resolving ambiguity: overriding the display method  
    public void display() {  
        System.out.println("Display method from class D");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        D obj = new D();  
        obj.display(); // Output: Display method from class D  
    }  
}
```

Q 31. Constructor VS Method in java?

	Feature	Constructor	Method
Purpose		Initializes the object.	Performs operations or actions on the object.
Called		Automatically called during object creation.	Called explicitly during the object's lifetime.
Return Type		Does not have a return type.	Has a return type (can be void or a data type).
Name		Must be the same as the class name.	Can have any name (except the class name).

Invoked	Invoked only once during object creation.	Can be invoked multiple times.
----------------	---	--------------------------------

Q 32. Java is a compiled language or interpreted language.

Java is both a compiled and interpreted language:

- **Compiled:** Source code is compiled to bytecode.
- **Interpreted:** Bytecode is executed by the JVM.
- **JIT Compilation:** The JVM may compile bytecode into native code at runtime for better performance.

Q 33. Can we override the static method?

No, we cannot override static methods in Java.

- **Static methods** are associated with the **class** rather than an **instance** (object) of the class.
- At compile time, the method to be called is determined by the class, not the object, so you cannot use dynamic polymorphism (overriding).
- Static methods can be **hidden** in subclasses (i.e., you can define a static method with the same name in a subclass), but this is not the same as overriding.

Q 34. Can we override the overloaded method?

- Yes, we can override an overloaded method in a subclass.
- Overloading and overriding are two different concepts:

- Overloading happens when methods in the same class have the same name but different parameter lists (either different numbers or types of parameters).
- Overriding happens when a subclass provides its own implementation of a method that is already defined in the superclass.
- Since overloading is based on method signatures (name + parameters), we can override a method in a subclass, provided that the method in the subclass has the same name and parameters as the method in the superclass.

Q 35. Can we overload the main() method?

Yes, we can overload the main() method in Java.

- The **main() method** is the entry point for Java programs. The signature of the main() method is public static void main(String[] args).
- You can **overload** the main() method by defining other main() methods with different parameter lists, just like any other method.
- However, when running the program, the JVM always calls the **public static void main(String[] args)** method. The other overloaded main() methods can be called explicitly in the code.

